



NLP Basics

 **github:** elma-dev

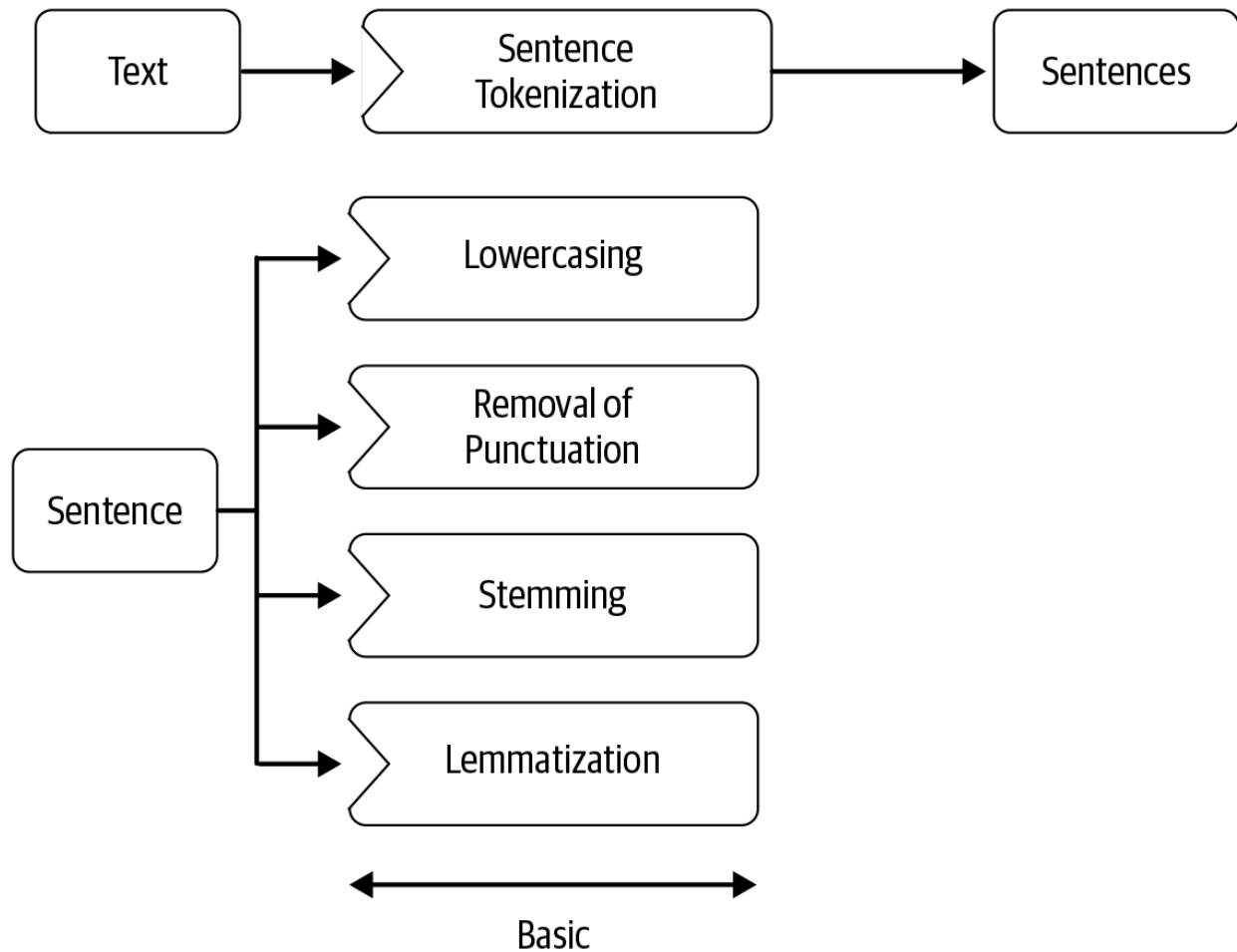
{ وَاتَّقُوا اللَّهَ وَبُعَلِّمُوا اللَّهَ وَاللَّهُ بِكُلِّ شَيْءٍ عَلِيمٌ }

 **linkden:** EL MAJJODI Abdeljalil

Data Preprocessing

A good practice to follow is to prepare a sequential list of pre-processing tasks to be done after having a clear understanding of how to process our data.

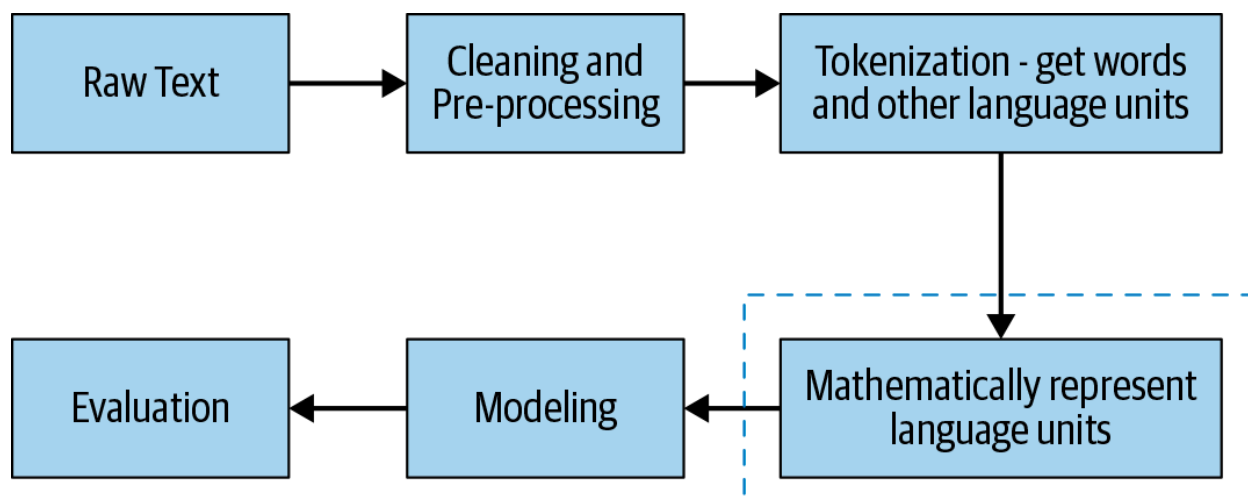
lists the different pre-processing steps:



Text Representation:

how do we go about doing feature engineering for text data?

⇒ In other words, how do we transform a given text into numerical form so that it can be fed into NLP and ML algorithms?



- Feature representation is a common step in any ML project, whether the data is text, images, videos, or speech.
- However, feature representation for text is often much more involved as compared to other formats of data.



What We See

08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
 49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 48 04 56 62 00
 81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
 52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
 22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
 24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
 32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
 67 26 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
 24 55 58 05 66 73 99 26 97 17 78 78 96 83 14 88 34 89 63 72
 21 36 23 09 75 00 76 44 20 45 35 14 00 61 33 97 34 31 33 95
 78 17 53 28 22 75 31 67 15 94 03 80 04 62 16 14 09 53 56 92
 16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
 86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
 19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
 04 52 08 83 97 35 99 16 07 97 57 32 16 26 26 79 33 27 98 66
 88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
 04 42 16 73 38 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
 20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
 20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 54
 01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 67 48

What Computers See

- We're given a piece of text, and we're asked to find a scheme to represent it mathematically. In literature, this is called text representation.

consider the following scenario: we're given a labeled text corpus and asked to build a sentiment analysis model. To correctly predict the sentiment of a sentence, the model needs to understand **the meaning of the sentence**. In order to correctly

extract the meaning of the sentence, the most crucial data points are:

1. Break the sentence into lexical units such as lexemes, words, and phrases
2. Derive the meaning for each of the lexical units
3. Understand the syntactic (grammatical) structure of the sentence
4. Understand the context in which the sentence appears

Vector Space Model:

- represent text units (characters, phonemes, words, phrases, sentences, paragraphs, and documents) with vectors of numbers. This is known as the **vector space model (VSM)**.
- It's a mathematical model that represents text units as vectors.
- the most common way to calculate the similarity between two text blobs using cosine similarity:

the cosine of the angle between their corresponding vectors. The cosine of 0° is 1 and the cosine of 180° is -1 , with the cosine monotonically decreasing from 0° to 180° . Given two vectors, A and B, each with n components, the similarity between them is computed as follows:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|_2 \|\mathbf{B}\|_2} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Basic Vectorization Approaches

One-Hot Encoding

We first map each of the six words to unique IDs: dog = 1, bites = 2, man = 3, meat = 4, food = 5, eats = 6. ii Let's consider document D1: "dog bites man". As per the scheme, each word is a six-dimensional vector. Dog is represented as [1 0 0 0 0 0], as the word "dog" is mapped to ID 1. Bites is represented as [0 1 0 0 0 0]

0 0], and so on and so forth. Thus, D1 is represented as [[1 0 0 0 0 0] [0 1 0 0 0 0] [0 0 1 0 0 0]]. D4 is represented as [[0 0 1 0 0] [0 0 0 0 1 0] [0 0 0 0 0 1]]. Other documents in the corpus can be represented similarly.

- **Shortcoming:**

- The size of one -hot vector is directly proportional to the size of the vocabulary, and most real-world corpora have a large vocabulary.
 - This results in a sparse representation where most of the entries in the vectors are zeros, making it computationally inefficient to store, compute with and learn from (sparsity leads to overfitting).
 - This representation does not give a fixed length representation for text, i.e., if a text has 10 words, you get a longer representation for it as compared to a text with 5 words. For most learning algorithms, we need the feature vectors to be of the same length.
 - It treats words as atomic units and has no notion of (dis)similarity between words. For example, consider three words: run, ran, and apple. Run and ran have similar meanings as opposed to run and apple. But if we take their respective vectors and compute the Euclidean distance between them, they're all equally apart $\sqrt{2}$. Thus, **semantically, they're very poor at capturing the meaning of the word in relation to other words.**

$\Rightarrow \text{run} = [1, 0, 0], \text{ran} = [0, 1, 0], \text{apple} = [0, 0, 1] \Rightarrow d(\text{run}, \text{ran}) = d(\text{run}, \text{apple})$

- Say we train a model using our toy corpus. At runtime, we get a sentence: "Man eats fruits." The training data didn't include "fruit" and there's no way to represent

it in our model. This is known as **the out-of-vocabulary (OOV)** problem.

Bag of words

The key idea behind it is as follows: **represent the text under consideration as a bag (collection) of words while ignoring the order and context.**

The basic intuition behind it is that it assumes that the text belonging to a given class in the dataset is characterized by a unique set of words.

If two text pieces have nearly the same words, then they belong to the same bag (class).

Similar to one-hot encoding, BoW maps words to unique integer IDs between 1 and $|V|$. Each document in the corpus is then converted into a vector of $|V|$ dimensions where in the **i th component of the vector, $i = \text{wid}$, is simply the number of times the word w occurs in the document**, i.e., we simply score each word in V by their occurrence count in the document.

```
corpus=np.array([
    "Dog bites man",
    "Man bites dog",
    "Dog eats meat.",
    "Man eats food."
])
our_vocabulary={'dog': 1, 'bites': 0, 'man': 4, 'eats': 2, 'food': 3}
bow(Dog bites man)=[[1 1 0 0 1]]
bow(Man bites dog)=[[1 1 0 0 1]]
count_vect.transform(["Man and Man are friends"]).toarray()
# array([[0, 0, 0, 0, 2]])
# has a value of 2 for the dimension of the word "dog," indic
```

ating its frequency
in the text.

- **Advantages:**

- Like one-hot encoding, BoW is fairly simple to understand and implement.
- With this representation, documents having **the same words will have their vector representations closer to each other in Euclidean space as compared to documents with completely different words**. The distance between D1 and D2 is 0 as compared to the distance between D1 and D4, which is 2. Thus, the vector space resulting from the BoW scheme **captures the semantic similarity** of documents.
So if two documents have similar vocabulary, they'll be closer to each other in the vector space and vice versa.
- We have a fixed-length encoding for any sentence of arbitrary length.

- **Disadvantages:**

- **The size of the vector increases with the size of the vocabulary. Thus, sparsity continues to be a problem**. One way to control it is by limiting the vocabulary to n number of the most frequent words.
- **It does not capture the similarity between different words that mean the same thing**. Say we have three documents: "I run", "I ran", and "I ate". BoW vectors of all three documents will be equally apart.
- **This representation does not have any way to handle out of vocabulary words** (i.e., new words that were not seen in the corpus that was used to build the vectorizer).

- As the name indicates, it is a “bag” of words—word order information is lost in this representation. Both D1 and D2 will have the same representation in this scheme.

Bag of N-Grams

All the representation schemes we’ve seen so far treat words as independent units.

breaking text into chunks of n contiguous words (or tokens).

This can help us capture some context, which earlier approaches could not do.

Each chunk is called an n -gram.

each document in the corpus is represented by a vector of length $|V|$.

⇒ This vector simply contains the frequency counts of n -grams present in the document and zero for the n -grams that are not present.

applicate 2_gram on our corpus:

```
{dog bites, bites man, man bites, bites dog,
 dog eats, eats meat, man eats, eats food}
```

The bigram representation for the first two documents is as follows:

```
D1 = [1,1,0,0,0,0,0,0]
```

```
D2 = [0,0,1,1,0,0,0,0]
```

```
# The results of using ngram_range=(1,3):
```

```
vocabulary: {'dog': 3, 'bites': 0, 'man': 9, 'dog bites': 4,
 'bites man': 2, 'dog bites man': 5, 'man bites': 10, 'bites
 dog': 1,
 'man bites dog': 11, 'eats': 6, 'food': 8, 'man eats': 12,
 'eats food': 7,
 'man eats food': 13}
```

- Note that the **BoW** scheme is a special case of the **BoN** scheme, with $n=1$.
- $n=2$ is called a “**bigram model**,” and $n=3$ is called a “**trigram model**.”

- **by increasing the value of n , we can incorporate larger context.**
- Here are the main **pros** and **cons** of BoN:
 - It **captures some context** and word-order information in the form of n -grams.
 - Thus, resulting vector space is **able to capture some semantic similarity**. Documents having the same n -grams will have their vectors closer to each other in Euclidean space as compared to documents with completely different n -grams.
 - As n increases, dimensionality (and therefore sparsity) only increases rapidly.
 - **It still provides no way to address the OOV problem.**

TF-IDF (*term frequency-inverse document frequency*)

all the words **in the text are treated as equally important**—there's no notion of some words in the document being more important than others.

TF-IDF, or *term frequency-inverse document frequency*, addresses this issue.

It aims to **quantify the importance of a given word relative to other words in the document and in the corpus**.

Concept:

- if a word w appears many times in a document d_i but does not occur much in the rest of the documents d_j in the corpus, then the word w must be of great importance to the document d_i . **(TF)**
- The importance of w should increase in proportion to its frequency in d_i , but at the same time, its importance should decrease in proportion to the word's frequency in other documents d_j in the corpus. **(ITF)**

TF:

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

measures how often a term or word occurs in a given document.

Since different documents in the corpus may be of different lengths, a term may occur more often in a longer document as compared to a shorter document. **To normalize these counts**, we divide the number of occurrences by the length of the document.

example:

```
len(d1)=100
count('word' in d1)=20
then TF('word', d1)=20/100= 0.2
-----
len(d2)=20
count('word' in d1)=4
then TF('word', d1)=4/20= 0.2
-----
TF('word', d1)=TF('word', d2)
```

IDF:

$$\text{IDF}(t) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

- measures the importance of the term across a corpus.
- In computing TF, all terms are given equal importance (weightage). However, it's a well-known fact that stop words like is, are, am, etc., are not important, even

though
they occur frequently.

→ IDF weighs down the terms that are very common across a corpus and weighs up the rare terms.

- the TF-IDF vectors to calculate similarity between two texts using a similarity measure like Euclidean distance or cosine similarity.
- TF-IDF is better than the vectorization methods we saw earlier in terms of capturing similarities between words.
- **it still suffers from the curse of high dimensionality.**

Distributed Representations

methods to learn low dimensional representations were devised.

They use neural network architectures to create dense, low-dimensional representations of words and texts.

Distribution Similarity:

- This is the idea that **the meaning of a word can be understood from the context** in which the word appears.
- also known as **connotation**: meaning is defined by context.
- opposed to **denotation**: the literal meaning of any word.

Distributional Hypothesis:

- In linguistics, this hypothesizes that words that occur in similar contexts have similar meanings.

For example, the English words "dog" and "cat" occur in similar contexts. Thus, according to the distributional hypothesis, there must be a strong similarity between the meanings of these two words.

- following from VSM, the meaning of a word is represented by the vector. Thus, if two words

often occur in similar context, then their corresponding representation vectors must also be close to each other.

Distributed Representation:

- the vectors in distributional representation are very high dimensional and sparse. This makes them computationally inefficient and hampers learning.
- **To alleviate this**, distributed representation schemes significantly compress the dimensionality.
- This results in vectors that are compact (i.e., low dimensional) and dense (i.e., hardly any zeros).

Word Embeddings

What does it mean when we say a text representation should capture **“distributional similarities between words”**? Let’s consider some examples. If we’re given the word “USA,” distributionally similar words could be other countries (e.g., Canada, Germany, India, etc.) or cities in the USA. If we’re given the word “beautiful,” words that share some relationship with this word (e.g., synonyms, antonyms) could be considered distributionally similar words.

- These are words that are likely to occur in similar contexts.

In 2013, a seminal work by Mikolov et al. [7] showed that their neural network–based word representation model known as “Word2vec,” based on “distributional similarity,” can capture word analogy relationships such as: King – Man + Woman ≈ Queen

While **learning such semantically rich relationships**, Word2vec **ensures that the learned word representations are low dimensional** (vectors of dimensions 50–500, instead of several thousands, as with previously studied representations in this chapter) and **dense** (that is, most values in these vectors are non-zero).

Word2Vec:

- takes a large corpus of text as input and “learns” to represent the words in a common vector space based on the contexts in which they appear in the corpus.

→ **Given a word w and the words appearing in its context C , how do we find the vector that best represents the meaning of the word?**

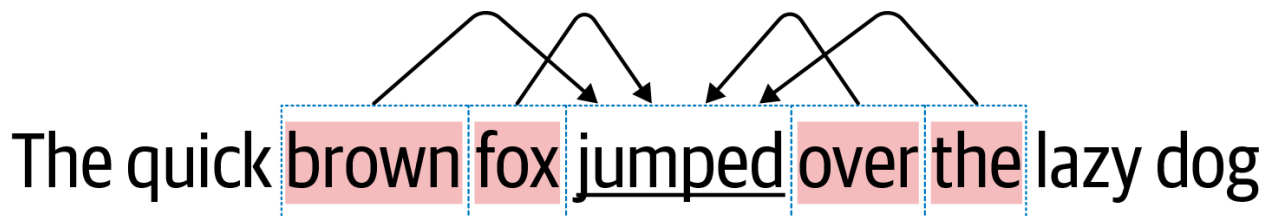
1. For every word w in corpus, we start with a vector V_w initialized with random values.
2. The Word2vec model **refines** the values in v_w by predicting v_w , given the vectors for words in the context C .

Training our own Embeddings:

For this, we'll look at two architectural variants that were proposed in the original Word2vec approach. The two variants are:

- ***CBOW: Continuous bag of words***

1. the primary task is to build a language model that correctly predicts the center word given the context words in which the center word appears.



What is a language model?

It is a (statistical) model that tries to give a probability distribution over sequences of words. Given a sentence of, say, m words, it assigns a probability $\Pr(w_1, w_2, \dots, w_n)$ to the whole sentence.

Details are as follows: we run a sliding window of size $2k+1$ over the text corpus. For our example, we took k as 2. Each position of the window marks the set of $2k+1$ words that are under consideration.

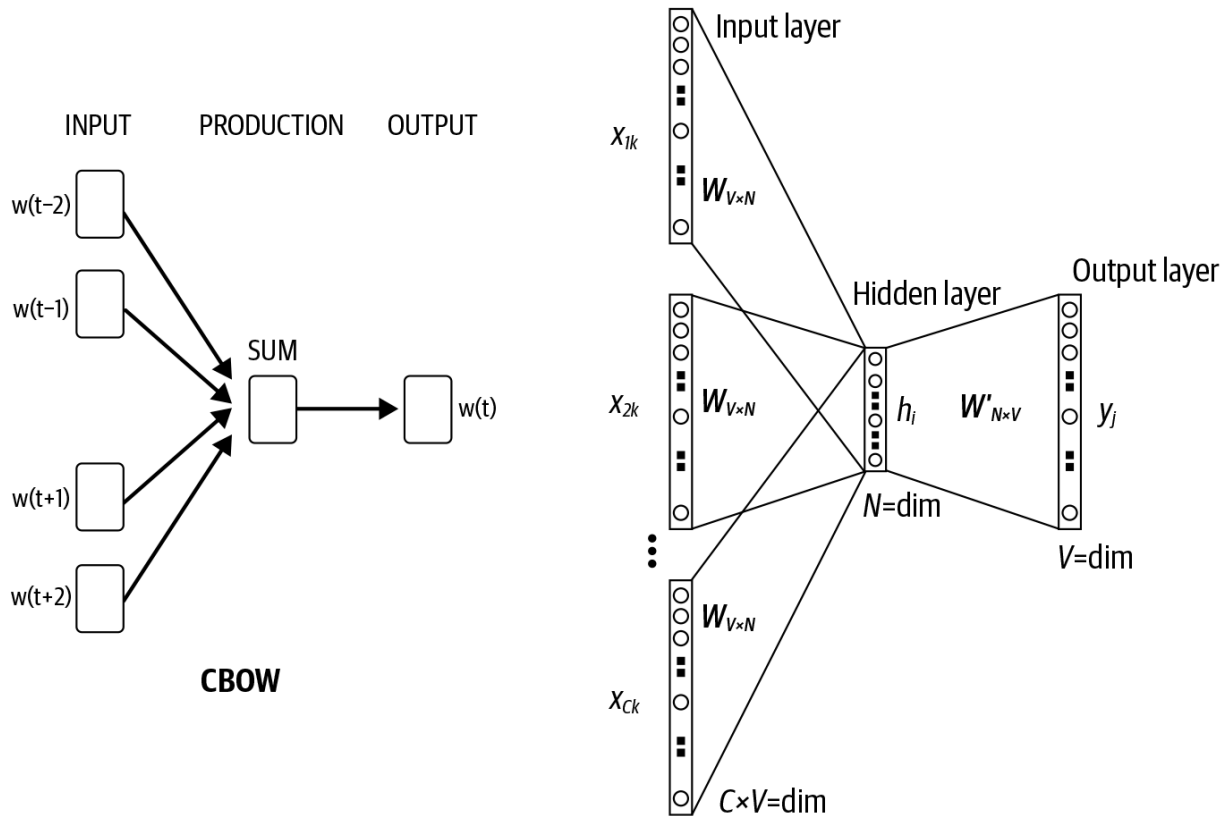
1. we run a sliding window of size $2k+1$ over the text corpus. For our example, we took k as 2. Each position of the window marks the set of $2k+1$ words that are under consideration.

2. **The center word in the window is the target**, and k words on either side of the center word form the context.
3. A single data point consists of a pair of numbers: (2k indices of words in context, index of word in target).
4. To get the next data point, we simply shift the window to the right on the corpus by one word and repeat the process.

Source Text	Training Samples (context, target)
The quick brown fox jumps over the lazy dog.	((quick, brown), The)
The quick brown fox jumps over the lazy dog.	((The, brown, fox), quick)
The quick brown fox jumps over the lazy dog.	((The, quick, fox, jumps), brown)
The quick brown fox jumps over the lazy dog.	((quick, brown, jumps, over), fox)

5. Create Shallow Model (has one hidden layer)

CBOW Model



- The objective is to learn an embedding matrix:

$$E_{|v|,d}$$

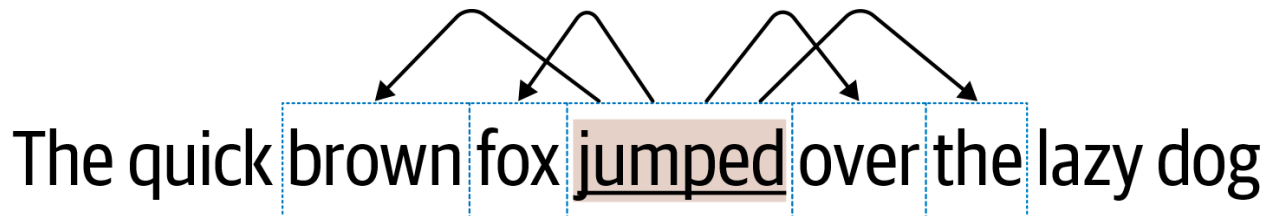
$v = \text{len}(\text{vocabulary})$
 $d : \text{dimension of Embeddings}$

- In the input layer, indices of the words in context are used to fetch the corresponding rows from the embedding matrix $E[|V| \times d]$.
- The next layer simply takes this d vector and multiplies it with another matrix $E'd \times |V|$.
- This gives a $1 \times |V|$ vector, which is fed to a softmax function to get probability distribution over the vocabulary space.

- This distribution is compared with the label and uses back propagation to update both the matrices E and E' accordingly.
- At the end of the training, E is the embedding matrix we wanted to learn.

- **Skip Gram:**

In **Skip-Gram**, the task is to predict the context words from the center word.



Details are as follows:

1. we run a sliding window of size $2k+1$ over the text corpus to get the set of $2k+1$ words that are under consideration.
2. The center word in the window is the X , and k words on either side of the center word are Y .
3. Unlike CBOW, this **gives us $2k$ data points**.
4. A single data point consists of a pair: (index of the center word, index of a target word) .
5. We then shift the window to the right on the corpus by one word and repeat the process.

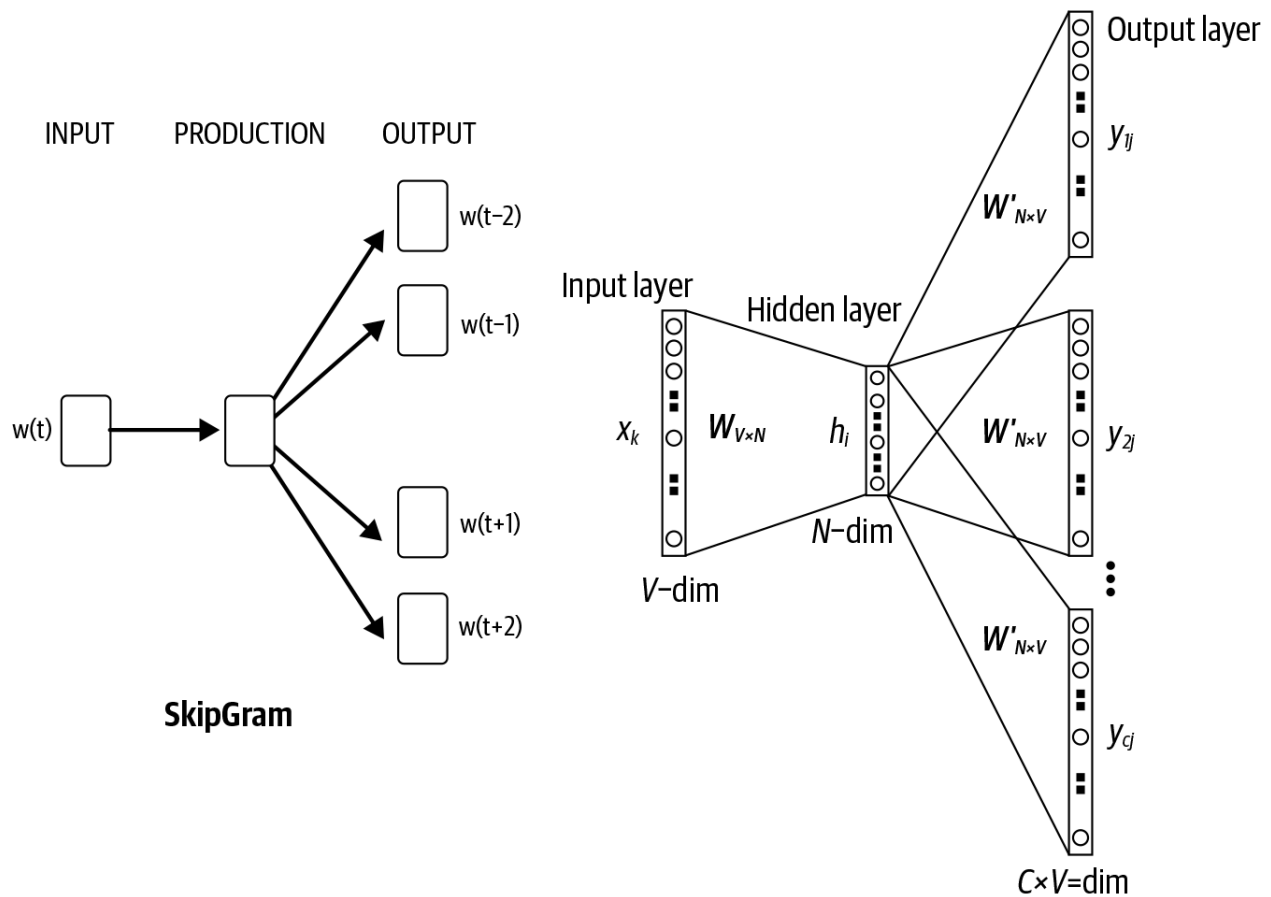
Source Text

Training Samples (target, context)

The quick brown fox jumps over the lazy dog.	→	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog.	→	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog.	→	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog.	→	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

6. Create Shallow Model (has one hidden layer):

Continuous SkipGram Model



Note that the output vectors (as well as the vectors u_c) are all identical, $y_1=y_2\ldots=y_C$

Visualizing Embeddings:

Unsupervised Learning

unsupervised learning refers to statistical methods that extract meaning from data without training a model on labeled data

Clustering methods can be used to identify meaningful groups of data.

→ For example, using the web clicks and demographic data of a user on a website, we may be able to group together different types of users.

- the goal may be to **reduce the dimension of the data** to a more manageable set of variables. This reduced set could then be used as input into a predictive model, such as regression or classification.
- Unsupervised techniques allow you to sift through and analyze these variables and discover relationships.

Principal Components Analysis

Principal components analysis (PCA) is a technique to discover the way in which numeric variables covary.

- variables will vary together (covary), and some of the variation in one is actually duplicated by variation in another (e.g., restaurant checks and tips).
- example:
 - imagine you have two things, like ice cream and the number of people at an ice cream shop. When more people come to the ice cream shop, the sales of ice cream might also go up because more people are buying it.
 - So, when we say "variables will vary together (covary)," it means that when one thing changes, the other thing tends to change too. In this case, when the number of people changes, the ice cream sales also change.